Practical Task - Full Stack Developer (MERN)

Overview

Build a **Job Board & Hiring Platform** - a complete multi-role web application where recruiters can post jobs, candidates can apply, and admins can monitor and manage the entire platform.

This task is designed to evaluate your full-stack development skills, architectural thinking, code quality, and ability to build a production-ready application.

Aspect	Requirement
Frontend	React.js or Next.js (TypeScript **strongly preferred**)
Backend	Node.js + Express (TypeScript **strongly preferred**)
Database	MongoDB
Authentication	JWT-based
Submission	GitHub repository + README.md

Note: TypeScript is **strongly preferred**. If you choose JavaScript, your code structure, consistency, and organization will be evaluated more deeply.

Roles

The application must support **three distinct roles** with proper authentication and role-based access control:

1. **Admin**
2. **Recruiter**
3. **Candidate**

Functional Requirements

1. Recruiter

- Register and log in
- Create, edit, delete, close, or reopen job listings
- Each job listing must include
    - title, description, required skills (array)
    - salary range, job type (Full-time, Part-time, Contract, etc.)
    - location, and application deadline
- View all applications received for each job

- Update application status per candidate (e.g., Applied, Shortlisted, Interviewed, Rejected, Hired)
- Personal dashboard showing active jobs count, total applicants, and recent activity

2. Candidate

- Register and log in
- Complete a basic profile: skills (array), years of experience, and resume upload
- Browse all open job listings with search and filter functionality (filters are your design decision)
- Apply to a job with a short application form - expected salary, availability, and a note to the recruiter
- Track the status of all their applications
- Save / bookmark jobs for later

3. Admin

- Overview dashboard with key platform-wide statistics
- Filter statistics by time period (Today, This Week, This Month, Custom Date Range)
- Visual charts showing trends (jobs posted over time, applications over time, top recruiters, etc.)
- View and manage all users (Recruiters and Candidates) - ability to activate/suspend accounts
- View all jobs and all applications across the platform (read-only access)

Bonus / Nice-to-Have Features

Implementing any of these will be considered a big plus **(not mandatory)**:

- Proper input validation using Zod
- Real-time notifications using Socket.io
- Advanced search with MongoDB text index or skills matching
- Email notifications on application status changes
- API documentation
- Basic unit or integration tests
- Docker setup

Additional information

What We Are Grading On

We care more about **quality, architecture, and best practices** than just completing all features. We will evaluate:

- **Code Architecture & Organization** (Frontend + Backend)
- **TypeScript usage** (types, interfaces, contracts, consistency)
- **Database Design** (Mongoose schemas, relationships, indexing, aggregation pipelines)
- **Authentication & Role-Based Access Control**
- **API Design** (RESTful structure, error handling, validation, response consistency)
- **Data Handling** (server-side filtering, pagination, search)
- **Frontend Architecture** (component design, state management, performance)
- **Code Quality & Readability** (naming, comments where needed, professionalism)
- **README.md** (setup instructions, environment variables, architectural decisions & trade-offs)

Submission Instructions

1. Push your complete code to a **public GitHub repository**
2. Your repository **must** include a detailed README.md with:
    - Step-by-step setup and run instructions
    - List of all required environment variables
    - Your architectural decisions and any trade-offs you made
    - List of bonus features implemented (if any)
3. (Optional but highly recommended) Add screenshots or a short screen recording video walkthrough of the working application

---

Important Note

Quality is significantly more important than quantity. Focus on clean code, good architecture, proper error handling, and a smooth user experience.

If you have any questions, feel free to reach out.

Good luck! We're excited to see your approach and code quality. 🚀